ESP8266 Robot Car

What changed from the original code to the modified code—and why

Purpose of this document: This guide compares your original sketch with the modified version. It explains the main changes intended to make HTTP commands get processed reliably, improve debugging, and stop the motors if commands stop arriving.

1. Quick summary

Area	Original code	Modified code	Why it matters
Command handling	Read `server.arg("State")` in `loop()` after handling clients.	Read and act on `State` inside `HTTP_handleRoot()`.	The request arguments are processed while the HTTP request is being handled.
Serial debugging	Mostly prints the AP IP; received commands are not clearly logged in the handler.	Prints request URI, received command, speed, startup messages, and timeout events.	Helps identify whether the phone/app is actually sending requests.
HTTP response	Returns an empty HTML response.	Returns a message for a valid command and an error for missing/unknown commands.	Makes browser testing and troubleshooting easier.
Stopping motors	Sets direction pins LOW but leaves PWM at `speedCar`.	Sets direction pins LOW and PWM to zero.	More explicitly disables motor-driver output in the stop routine.
Command timeout	No automatic stop if commands stop arriving.	Stops after 1 second without a fresh movement command.	Adds a basic safety fallback; the app may need to repeat movement requests.
Speed commands	Changes the global speed variable directly.	Uses `setSpeed()` to change speed and print it.	Keeps speed handling in one place and makes it easier to debug.

2. The most important change: where commands are processed

In the original sketch, `loop()` called `server.handleClient()` and then read `server.arg("State")`. The HTTP request and its arguments belong to the request-handling process, so reading the argument later in the main loop is not a reliable way to process each request.

Original structure

```
void loop() {
  server.handleClient();
  command = server.arg("State");
  if (command == "F") goAhead();
  // other commands...
}
```

Modified structure

```
void HTTP_handleRoot() {
  if (server.hasArg("State")) {
    command = server.arg("State");
    // execute the matching movement/speed command here
    server.send(200, "text/plain",
                "Command received: " + command);
  } else {
    server.send(400, "text/plain",
                "Missing State parameter");
  }
}
```

```
void loop() {
  server.handleClient();
  // check the safety timeout
}
```

Why this helps: When the phone opens a request such as `/?State=F`, the server calls `HTTP_handleRoot()`. That handler reads the `State` parameter and immediately performs the requested action. The main loop no longer tries to retrieve a command after request handling has finished.

3. Added serial messages for diagnosis

The modified code prints useful status messages to the Serial Monitor at 115200 baud, including:

• Robot startup and Wi-Fi access-point name
• Access-point IP address
• HTTP request received and request URI
• The exact command received (F, B, L, R, S, 0–9, etc.)
• Current speed value
• A message when the safety timeout stops the motors

These messages help separate a Wi-Fi connection problem from an HTTP/app problem. If the phone connects to the car's Wi-Fi but no command appears in Serial Monitor, the app/browser may not be sending the expected request.

4. Added clear HTTP replies

The original handler sent an empty HTML page. The modified handler replies with readable text for valid commands and sends HTTP status 400 when the State parameter is missing or the command is unknown.

Example browser test

```
http://192.168.4.1/?State=F
http://192.168.4.1/?State=S
```

The first request asks the car to move forward; the second asks it to stop. The exact IP address should be confirmed in the Serial Monitor. A phone app must send requests in the same format, or the handler will report a missing/unknown command.

5. Changed the stop routine

Original code:

```
void stopRobot() {
  digitalWrite(IN_1, LOW);
  digitalWrite(IN_2, LOW);
  analogWrite(ENA, speedCar);
  digitalWrite(IN_3, LOW);
  digitalWrite(IN_4, LOW);
  analogWrite(ENB, speedCar);
}
```

Modified code:

```
void stopRobot() {
  digitalWrite(IN_1, LOW);
  digitalWrite(IN_2, LOW);
  digitalWrite(IN_3, LOW);
  digitalWrite(IN_4, LOW);
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
  motorsMoving = false;
}
```

The modified routine sets both motor-direction inputs LOW and sets both enable PWM values to zero. This explicitly requests no motor drive from the L298N. Depending on the driver and wiring, this is generally a coast/no-drive stop rather than guaranteed active braking.

6. Added a 1-second safety timeout

New variables:

```
unsigned long lastCommandTime = 0;
const unsigned long commandTimeout = 1000;
bool motorsMoving = false;
```

Every movement function updates `lastCommandTime` and marks the motors as moving. The main loop checks elapsed time. If no fresh movement command arrives for 1000 milliseconds (1 second), it calls `stopRobot()`.

```
if (motorsMoving &&
    millis() - lastCommandTime >= commandTimeout) {
  stopRobot();
  Serial.println("Safety Timeout: Motors Stopped");
}
```

Important: This timeout can make the car stop after one second if the control app sends only one movement request when you press a button. For continuous motion, the app should repeat movement commands while the button is held. Test with the wheels lifted off the ground first.

7. Added a speed-setting function

Original code directly changed `speedCar` for each digit command. The modified code calls a helper function:

```
void setSpeed(int newSpeed) {
  speedCar = newSpeed;
  if (motorsMoving) {
    analogWrite(ENA, speedCar);
    analogWrite(ENB, speedCar);
  }
  Serial.print("Speed: ");
  Serial.println(speedCar);
}
```

This keeps speed updates in one function and prints the new value. One limitation in this version is that it sets both motor PWM values equally if the car is already moving; during a diagonal movement, that can remove the reduced-speed difference between the two sides until the next movement command.

8. What did not change?

The modified code keeps the same main hardware mapping and overall design:

• Same ESP8266 access-point mode and Wi-Fi name: `NodeMCU Car`
• Same web server port: 80
• Same motor pins: ENA GPIO14 (D5), ENB GPIO12 (D6), IN_1 GPIO15 (D8), IN_2 GPIO13 (D7), IN_3 GPIO2 (D4), IN_4 GPIO0 (D3)
• Same basic command letters for forward, backward, left, right, diagonals, and stop
• Same speed range of 400–1023

The changes mainly concern request processing, debugging, motor stopping, and safety behavior—not a redesign of the robot's Wi-Fi or motor wiring.

9. How to verify that commands are reaching the code

1. Upload the modified sketch and open Serial Monitor at 115200 baud.
2. Find the printed access-point IP address (commonly `192.168.4.1`).
3. Connect the phone to the Wi-Fi network `NodeMCU Car`.
4. In a browser, visit `http://192.168.4.1/?State=F`.
5. Confirm that Serial Monitor shows an HTTP request and `Received Command: F`.
6. Send `http://192.168.4.1/?State=S` to stop.
7. Only after browser testing works, test the app and confirm it sends the same request pattern.

10. Hardware and testing cautions

• Lift the wheels off the ground for first tests.
• Use a suitable separate supply for the motors and L298N; do not power motors directly from the NodeMCU 3.3 V pin.
• Connect NodeMCU ground and motor-driver ground together.
• ESP8266 pins GPIO0, GPIO2, and GPIO15 affect boot mode. Ensure the driver/wiring does not force them into the wrong states during startup.
• The modified code is designed to improve command handling, but it does not prove the app is sending valid requests or that the wiring/power is correct.

11. Bottom line

The key software fix is to read and execute the State command inside the HTTP request handler instead of reading it later in loop(). The added serial logs and HTTP replies make failures easier to diagnose. The zero-PWM stop routine and timeout add a basic safety fallback. Whether the car now works still depends on the app's request format, motor-driver wiring, power, and the specific ESP8266 board.